PROJECT REPORT

Steel Surface Defect Classification

Submitted by

Daksh Bawa (1025150368)

Nandasuta Jagota (1025150367)

Priyal Gupta (1025150444)

Somya Sekhri (1025150382)

Tanisha (1025150393)

Submitted to

Ms. Jyoti



Figure 1:

Electronics and Computer Engineering

Thapar Institute of Engineering and Technology, Patiala

Academic Year 2026–2027

Contents

1 Abstract . . . 3
2 Introduction . . . 3
2.1 Background . . . 3
2.2 Problem Statement . . . 3
2.3 Objectives . . . 4
2.4 Scope . . . 4
3 Dataset Description . . . 4
3.1 Dataset Source . . . 4
3.2 Defect Categories . . . 4
3.3 Exploratory Data Analysis . . . 6
4 Methodology . . . 6
4.1 Workflow Overview . . . 6
4.2 EDA and Key Observations . . . 6
4.3 Image Preprocessing . . . 7
4.4 Data Augmentation . . . 7
4.5 Custom CNN Architecture . . . 7
4.6 Training Controls . . . 8
4.7 Feature Selection . . . 8
5 Model Evaluation . . . 8
5.1 Evaluation Metrics . . . 8
5.2 Evaluation Procedure . . . 8
5.3 Evaluation and Model Comparison . . . 9
5.4 Interpretation of Results . . . 9
5.5 Interpreting the Confusion Matrix . . . 11
5.6 Training Curves . . . 11
6 Explainable AI Using Grad-CAM . . . 11
7 Model Optimization with TensorFlow Lite . . . 12
7.1 TensorFlow Lite Optimization . . . 12
8 Streamlit Application and Deployment . . . 12
8.1 Application Functionality . . . 12
8.2 Deployment Requirements . . . 12
8.3 Deployment Validation . . . 13

9 Ethical Considerations, Limitations, and Risks 14
9.1 Limitations 14
9.2 Responsible Use 14
9.3 Future Scope 14
10 Conclusion 15
11 Appendix A 15
12 Website and App 15
13 Github 15

1 Abstract

Surface defects in steel can affect product quality, manufacturing efficiency and the reliability of downstream components. Manual inspection is useful but can be time-consuming and may vary between inspectors. This project develops an image-classification prototype that uses deep learning to classify steel surface images into six defect categories: crazing, inclusion, patches, pitted surface, rolled-in scale and scratches.

The implementation uses the NEU surface-defect dataset accessed through KaggleHub. Two approaches are explored: a custom convolutional neural network (CNN) trained from scratch on grayscale images and MobileNetV2 with ImageNet-pretrained weights used as a transfer-learning backbone for RGB images. The notebook includes dataset exploration, image resizing and normalization, training-only data augmentation, validation-based callbacks, classification metrics, confusion matrices, Grad-CAM visualization, TensorFlow Lite conversion and a Streamlit image-upload interface.

The purpose of the project is to compare a compact custom CNN with a pretrained convolutional architecture and to demonstrate a path from model development to an interpretable prototype application. Final numerical performance values should be inserted from the corrected, final run of the notebook. In particular, MobileNetV2 preprocessing and the dashboard input pipeline must be checked for consistency before reporting final results.

2 Introduction

2.1 Background

Steel is used in construction, transportation, machinery, appliances and many other industrial applications. During production, handling, rolling or finishing, the steel surface may develop visible irregularities. Detecting such defects early can help maintain quality and reduce the risk of defective material moving to later production stages.

Computer vision systems can learn visual patterns from labelled images and predict the category of a surface defect. Convolutional neural networks are suited to image tasks because convolutional filters learn local features such as edges, textures, lines and patterns. Transfer learning can further reduce training requirements by reusing features learned from a large image dataset.

2.2 Problem Statement

The project addresses a six-class image-classification problem: given an image of a steel surface, predict which of the six labelled defect categories it belongs to. The model receives a resized image and returns a probability score for each class. The class with the highest score is presented as the predicted category.

2.3 Objectives

- Explore the dataset, identify the defect classes and inspect representative images.
- Prepare training, validation and held-out evaluation datasets.
- Build and train a custom CNN baseline.
- Build a MobileNetV2 transfer-learning model and compare its performance with the baseline.
- Evaluate predictions using accuracy, macro precision, macro recall, macro F1-score, classification reports and confusion matrices.
- Use Grad-CAM to visualize image regions that influence a model prediction.
- Convert the transfer-learning model to TensorFlow Lite and create a Streamlit demonstration application.
- Discuss deployment considerations, limitations and possible future improvements.

2.4 Scope

The work is a prototype for image-based defect classification. It demonstrates a model-development and deployment workflow; it is not a certified industrial inspection system. The notebook does not establish performance on every steel grade, camera, lighting condition, production line or real-time sensor environment.

3 Dataset Description

3.1 Dataset Source

The notebook downloads the public NEU surface-defect database through the KaggleHub dataset identifier 'kaustubhdikshit/neu-surface-defect-database'. It searches the downloaded directory for a training image folder and a held-out validation image folder, then loads images by their class subdirectories.
The exact image counts should be copied from the notebook output after the final run. They are intentionally not fabricated in this report because the attached notebook source does not include reliable printed output values.

3.2 Defect Categories

graphicx float placeins

Dataset component	Use in this project	Important note
Training subset	Used to fit model weights	An 80% / 20% split is created from the training folder.
Validation subset	Used during training to monitor generalization and callbacks	Derived from the training folder using a fixed seed.
Held-out folder	Used for final model evaluation	Kept separate from the training/validation split in the intended workflow.

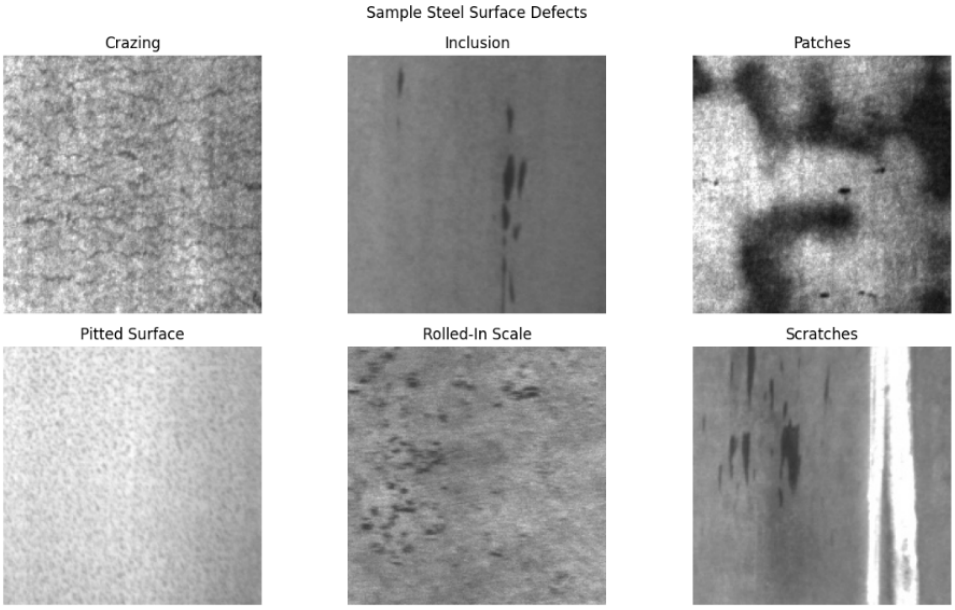


Figure 2: Sample steel surface defects

Class label in code	General visual interpretation
crazing	Fine network-like cracks or crack patterns on the surface.
inclusion	Foreign or non-metallic material-related surface indications.
patches	Patch-like surface texture or appearance variation.
pitted_surface	Small pit-like depressions or marks.
rolled-in_scale	Scale-related surface features associated with rolling.
scratches	Linear marks or grooves on the surface.

3.3 Exploratory Data Analysis

The notebook counts supported image files in each class directory and displays the training and held-out image counts in a grouped bar chart. It also displays one representative training image for each class and plots a sample batch after preprocessing.
For the final submission, include the generated dataset-distribution chart and sample-image grid here. Add a short evidence-based interpretation, for example whether class counts are similar, which classes have fewer samples, and whether some defect textures appear visually similar. Do not claim that the dataset is balanced unless the counts support that conclusion.

4 Methodology

4.1 Workflow Overview

Stage	**Description**
1. Data acquisition	Download the dataset and locate class-organized folders.
2. EDA	Count images, inspect class distribution, and display representative samples.
3. Preprocessing	Resize images and create model-specific input pipelines.
4. Model development	Train a custom CNN and a MobileNetV2 transfer-learning model.
5. Evaluation	Calculate metrics and inspect confusion matrices on held-out data.
6. Explainability	Generate Grad-CAM heatmaps for selected predictions.
7. Optimization	Convert the transfer model to TensorFlow Lite with default optimization.
8. Demonstration	Use a Streamlit interface to upload an image and display a predicted class.

4.2 EDA and Key Observations

Record the dataset totals and class counts, comment on whether the distribution is balanced, describe visible differences among the six classes and note any visual similarities that may make classification difficult. Complete these observations from the notebook output; do not guess counts or patterns.

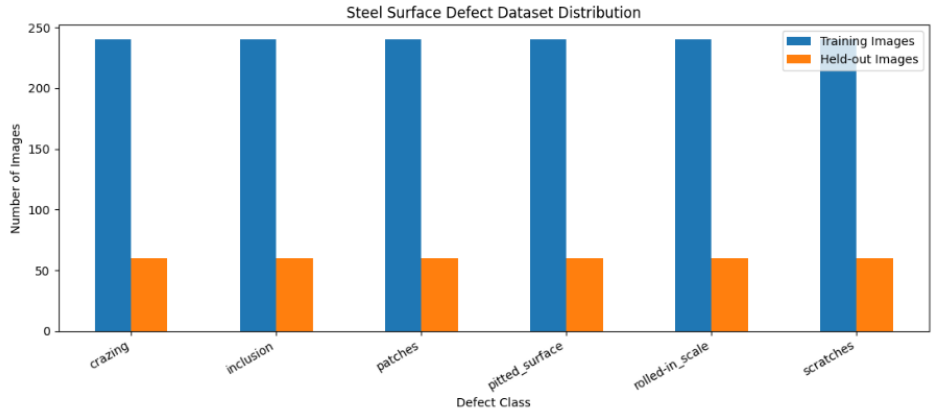


Figure 3: Dataset Class Distribution

4.3 Image Preprocessing

For the custom CNN, images are loaded in grayscale and pixel values are converted to floating-point values in the range 0 to 1 by dividing by 255. The model input shape is $128 \times 128 \times 1$.

For MobileNetV2, images are loaded in RGB at 160×160 pixels. MobileNetV2's application-specific `preprocess_input` maps pixel values to the range expected by the pretrained network. This transformation must be applied exactly once.

The supplied notebook currently applies preprocessing in the dataset pipeline and also includes preprocessing inside the model, with an additional multiplication by 255. Before final results are reported, this must be corrected so the model and data pipeline have a single, consistent preprocessing path. The Streamlit application must use the same input convention as the corrected model.

4.4 Data Augmentation

The notebook uses random rotation, translation, and zoom as data augmentation. Augmentation is intended to expose the model to modest variations in image orientation and framing, reducing over-reliance on the exact appearance of training samples. Augmentation should be active only during training and should not alter validation or test images at evaluation time.

4.5 Custom CNN Architecture

The custom CNN accepts a single-channel 128×128 image. It applies three convolutional blocks with 32, 64, and 128 filters respectively, each using 3×3 kernels, same padding, and ReLU activation. Each convolutional block is followed by max pooling. The final feature maps are reduced using global average pooling. A dropout layer with rate 0.35 is followed by a 64-unit dense layer with ReLU activation and L2 regularization of 0.001, then a second dropout layer with rate 0.25. A final dense layer with six softmax outputs produces class probabilities.

Component	Purpose
Input	128 × 128 × 1 grayscale image
Convolutional blocks	32, 64, and 128 filters; 3 × 3 kernels; ReLU
Downsampling	Max pooling after each convolution
Feature aggregation	Global average pooling
Regularization	Dropout rates 0.35 and 0.25; L2 penalty 0.001
Classifier	Dense 64 units + six-unit softmax output
Optimizer / loss	Adam, learning rate 0.001; sparse categorical cross-entropy

4.6 Training Controls

Both training workflows use callbacks to reduce unnecessary training and retain a strong checkpoint. Early stopping monitors validation loss and restores the best weights after the configured patience period. ReduceLROnPlateau reduces the learning rate when validation loss stops improving. ModelCheckpoint saves the model with the best validation loss. The CNN is configured for up to 30 epochs with patience 5; MobileNetV2 is configured for up to 15 epochs with patience 4. The actual completed epoch count should be copied from the final training output.

4.7 Feature Selection

This project learns visual features automatically rather than using a separate hand-crafted feature-selection algorithm. The custom CNN learns patterns such as edges, textures, and defect boundaries through convolutional layers. MobileNetV2 uses pretrained visual representations, followed by global average pooling and a six-class classification head.

The main feature-preparation steps are image resizing, channel selection (grayscale for the custom CNN and RGB for MobileNetV2), normalization or model-specific preprocessing, and training-only augmentation. Do not claim that a separate statistical feature-selection technique was implemented unless you add and evaluate one.

5 Model Evaluation

5.1 Evaluation Metrics

5.2 Evaluation Procedure

The notebook uses the held-out images to obtain model probabilities and selects the class with the largest probability using ‘argmax‘. It calculates accuracy, macro precision, macro recall, macro F1-score, and a per-class classification report. Confusion matrices are plotted as heatmaps. The notebook also includes a faster comparison that evaluates up to 300 images, as well as cells that evaluate the complete held-out dataset.

Metrics	Meaning
Input	Fraction of all evaluated images whose predicted class matches the true class.
Precision	Among predictions assigned to a class, the fraction that are correct.
Recall	Among actual examples of a class, the fraction correctly identified.
F1-score	Harmonic mean of precision and recall.
Macra Average	Compute a metric separately for each class and average the class scores equally.
Confusion matrix	Table showing actual classes against predicted classes, revealing correct predictions and class-to-class errors.

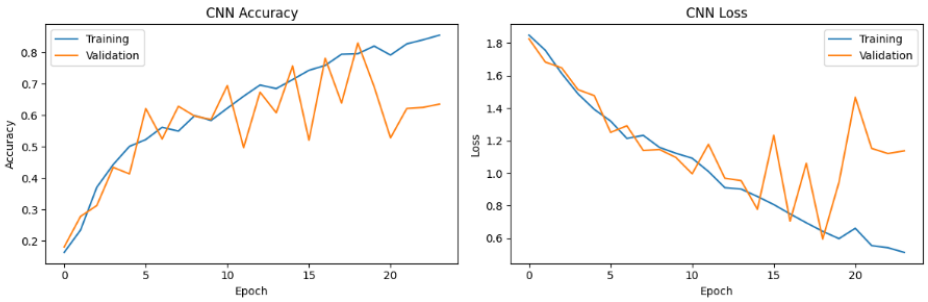


Figure 4: CNN training and validation curves

For the final report, use the complete held-out set for the headline metrics of both models, and ensure that the same image set is used for a fair comparison. Do not mix results from a 300-image subset with results from the full set in one comparison table.

5.3 Evaluation and Model Comparison

Compare both models on the same complete held-out test set, using accuracy, macro precision, macro recall, macro F1-score, per-class recall, model size, and inference latency. Use the confusion matrices to identify which defect categories are most often confused. Choose the final model based on the measured results rather than assuming that one architecture is automatically better.

5.4 Interpretation of Results

In this section, explain what the results mean rather than simply listing scores. State which model performs best overall, which class has the lowest recall, what the largest confusion-matrix errors suggest, whether the training curves show overfitting, whether Grad-CAM focuses on the visible defect, and whether TensorFlow Lite improves size or speed without unacceptable quality loss. Support every claim with a table, graph, or measured value from the final run.

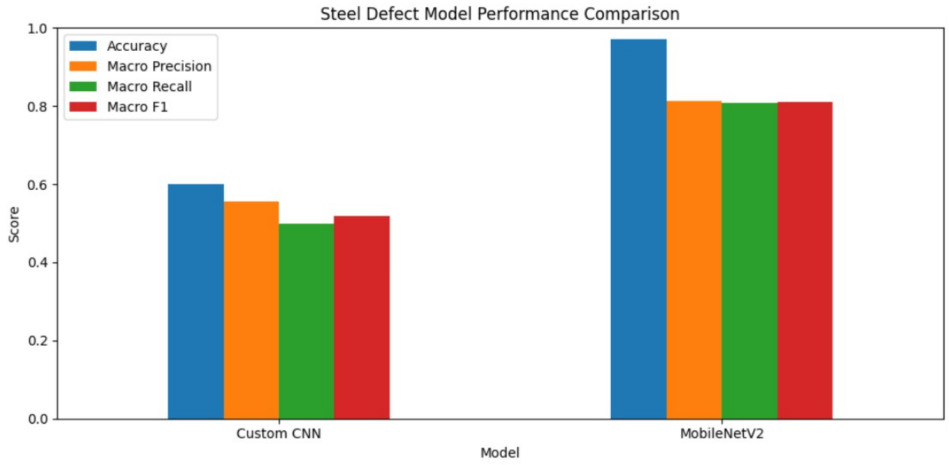


Figure 5: Model performance comparison

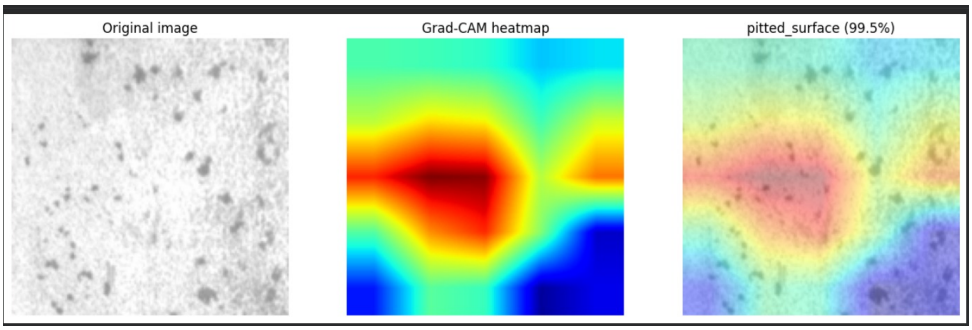


Figure 6: Grad-CAM Visualization

5.5 Interpreting the Confusion Matrix

The diagonal of a confusion matrix represents correct predictions. Off-diagonal values represent misclassifications. Large off-diagonal counts indicate classes the model confuses more frequently. In the final report, identify the two or three largest confusions from the generated matrix and discuss whether the classes share similar textures or visual patterns. Also inspect per-class recall to identify defect types the model may miss.

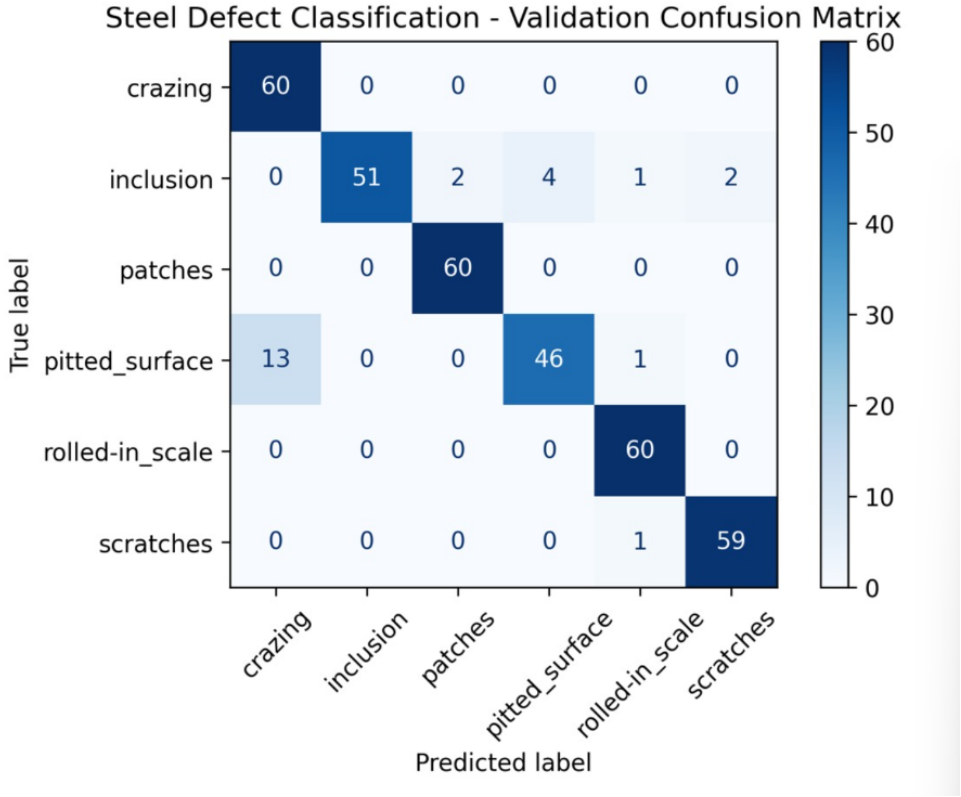


Figure 7: Confusion Matrix

5.6 Training Curves

The notebook plots training and validation accuracy and loss for the custom CNN. These curves can help identify underfitting or overfitting. A large gap between training and validation performance can suggest overfitting; low performance on both can suggest underfitting or an inadequate training setup. The interpretation should be based on the actual plotted curves.

6 Explainable AI Using Grad-CAM

Gradient-weighted Class Activation Mapping (Grad-CAM) provides a visual explanation of a CNN prediction by using gradients of the selected class score with respect to convolutional feature maps. The gradients are pooled to produce weights for the feature channels. A weighted combination of the feature maps is passed through a ReLU and normalized to create a heatmap.

The notebook applies Grad-CAM to the MobileNetV2 feature extractor, resizes the heatmap to the input image size, and displays three views: the original image, the heatmap, and an overlay. The predicted class and confidence are shown with the visualization.
Interpretation must be cautious. Warm or high-intensity regions indicate image areas that contributed more strongly to the selected class score in this visualization; they do not prove that the model's reasoning is correct. A heatmap that focuses on background, borders, or irrelevant regions can reveal a potential weakness. The report should include multiple defect classes and, if possible, one misclassified example.

7 Model Optimization with TensorFlow Lite

7.1 TensorFlow Lite Optimization

The notebook converts the trained transfer-learning model to TensorFlow Lite using `TFLiteConverter.fro` and enables `tf.lite.Optimize.DEFAULT`. The resulting model is saved as `steel_defect_model.tflite`.

This is a useful deployment step because TensorFlow Lite is designed for inference in resource-constrained environments.

Conversion alone does not demonstrate that the optimized model is faster or equally accurate. For a complete optimization evaluation, record the Keras model size and TFLite model size, measure average inference latency over repeated runs under the same hardware conditions and compare TFLite predictions with the original model. If any prediction differences occur, report them honestly.

8 Streamlit Application and Deployment

8.1 Application Functionality

The notebook writes a Streamlit application that allows a user to upload a JPG, JPEG, or PNG image. The app converts the image to RGB, resizes it to 160×160 pixels, adds a batch dimension, and passes it to the saved MobileNetV2 model. It displays the predicted defect category and the model's highest class probability. One version of the app also displays the probability scores for all classes.

8.2 Deployment Requirements

The model file and application file must be in the expected locations. The notebook currently uses Google Colab paths such as `/content/steel_defect_model.keras`; these paths will not automatically exist on another computer.

For a portable submission, use a project folder with a consistent relative path, provide a requirements file, and document how to run `streamlit run app.py`.

Suggested project item	Purpose
`app.py`	Streamlit application
`steel_defect_model.keras`	Final Keras model
`steel_defect_model.tflite`	Optimized model, if included
`requirements.txt`	Python package dependencies
`README.md`	Installation, usage, and troubleshooting
`project_notebook.ipynb`	Training, evaluation, and explanation workflow
`results/`	Final charts, confusion matrices, Grad-CAM images, and metrics

8.3 Deployment Validation

- Test the app with images from each of the six supported categories.
- Check that the class order in the app exactly matches the training class order.
- Confirm that the app's preprocessing matches the corrected model input pipeline.
- Check behaviour for invalid files, corrupted images, and images that are unlike the training data.
- Verify that the saved model loads from a fresh environment with documented dependencies.
- Display a clear warning that predictions are experimental and require human verification.

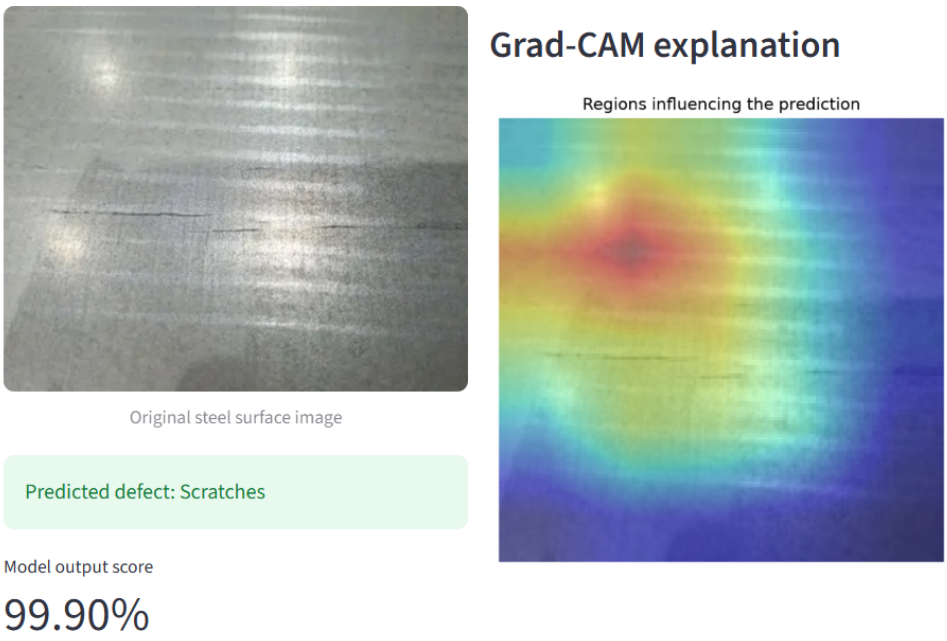


Figure 8: Streamlit Application Screenshot

9 Ethical Considerations, Limitations, and Risks

9.1 Limitations

- The model is trained on a particular public dataset and may not generalize to every factory, steel type, camera, lighting setup, or image resolution.
- The six-class system can only predict among its known labels; it is not designed to reliably detect unknown defect types.
- Confidence values from a softmax layer are not guaranteed to be calibrated probabilities of correctness.
- Incorrect predictions can lead to missed defects or unnecessary rejection of acceptable material if the prototype is used without verification.
- Grad-CAM is a diagnostic visualization, not a proof of causality or correctness.
- Performance should be re-evaluated on representative production-line images before operational use.

9.2 Responsible Use

Any industrial deployment should retain human oversight, define acceptable error rates, monitor changes in image quality and defect prevalence, and periodically re-evaluate the model. Images and production information should be handled according to the organization's privacy and data-retention policies. The system should not be described as replacing qualified quality-control personnel based solely on results from a curated dataset.

9.3 Future Scope

- Fine-tune selected MobileNetV2 layers using a lower learning rate after training the classifier head.
- Evaluate additional architectures under the same train/validation/test protocol.
- Add a prediction API or a simulated image-stream input for integration testing.
- Add a confidence threshold and an 'uncertain—manual review required' outcome.
- Create a dashboard showing class-wise metrics, error examples, model version, and inference latency.
- Benchmark model quantization or other optimization methods and validate accuracy after conversion.
- Evaluate on real production images and monitor performance drift over time.

10 Conclusion

This project presents an end-to-end prototype for steel surface defect classification using deep learning. It combines exploratory data analysis, a custom CNN baseline, MobileNetV2 transfer learning, quantitative evaluation, confusion-matrix visualization, Grad-CAM explainability, TensorFlow Lite conversion, and a Streamlit image-upload interface. These components address the main stages of an applied machine-learning workflow from dataset preparation to a user-facing demonstration.

The final claims about model quality must be based on a corrected and reproducible evaluation. In particular, preprocessing should be applied consistently, both models should be evaluated on the same complete held-out set, and optimization claims should be supported by measured size and latency comparisons. With these checks, documented limitations, and a reproducible deployment package, the project can be presented as a responsible proof of concept for automated surface-defect classification.

11 Appendix A

- Insert actual image counts and class distribution chart.
- Insert representative sample images for all six classes.
- Insert training/validation accuracy and loss plots.
- Insert full-test confusion matrices for both models.
- Fill the final metrics table using a corrected, complete held-out evaluation.
- Insert Grad-CAM original/heatmap/overlay and interpretation.
- Verify the app loads the corrected model with consistent preprocessing.

12 Website and App

APP- https://steelguard-ai-euatks776jsjkxvt664hnd.streamlit.app/
WEBSITE- https://steelguard-ai-euatks776jsjkxvt664hnd.streamlit.app

13 Github

GITHUB- https://github.com/dbawabe25-afk/steelguard-ai